

Customer Churn Prediction: A Comparative Study of XGBoost and MLP Classifiers

Shashen Perera

August 16, 2026

1 Introduction

Customer churn is a critical metric for subscription-based businesses. This report presents a comparative analysis of three machine learning classifiers applied to a customer churn dataset: XGBoost, a Keras-based Multilayer Perceptron (MLP), and a Scikit-learn MLP.

The use of two different MLP implementations was intentional. While the course notes used TensorFlow / Keras primarily for neural networks, the literature suggests that for moderate-sized datasets (e.g. $\sim 5,000$ samples), the Scikit-learn MLPClassifier can be more stable and easier to tune due to its built-in adaptive learning rate - and early stopping capabilities. Moreover, it requires less boilerplate code and is often faster to experiment with. However, Keras offers greater flexibility for custom architectures and is more suitable when integration with TensorFlow pipelines is needed. Therefore, we decided to implement and optimize both, comparing their performance and ease of use.

2 Dataset and Preprocessing

2.1 Dataset Overview

The dataset used in this study is a customer churn dataset sourced from Kaggle, containing 5,000 customer records. It includes 12 features: `Customer_ID`, `Age`, `Gender`, `Subscription_Length`, `Region`, `Payment_Method`, `Support_Tickets_Raised`, `Satisfaction_Score`, `Discount_Offered`, `Last_Activity`, `Monthly_Spend`, and `Churned` (binary target). The dataset exhibits a moderate class imbalance, with 44.8% churned and 55.2% retained customers.

2.2 Data Cleaning and Transformation

The following preprocessing steps were applied:

- **Missing Values:** `Age` and `Satisfaction_Score` each had 10% missing values. Both features showed near-normal distributions (skewness ≈ 0) with no significant outliers, so mean imputation was used.
- **Feature Encoding:** Categorical variables (`Gender`, `Region`, `Payment_Method`) were one-hot encoded using `pd.get_dummies` with `drop_first=True` to avoid multicollinearity.
- **Train-Test Split:** Data was split into 80% training and 20% testing using stratified sampling to preserve class proportions.
- **Feature Scaling:** For MLP models, `StandardScaler` was applied within a pipeline to avoid data leakage.

2.3 Exploratory Data Analysis

Key EDA insights:

- Correlation analysis revealed that `Monthly_Spend` and `Satisfaction_Score` have strong negative correlations with churn, while `Support_Tickets_Raised` shows a positive correlation.
- Box plots showed that churned customers tend to have lower monthly spend, lower satisfaction scores, and higher support ticket counts.
- Churn rates varied slightly across regions and payment methods, but gender showed no significant difference.

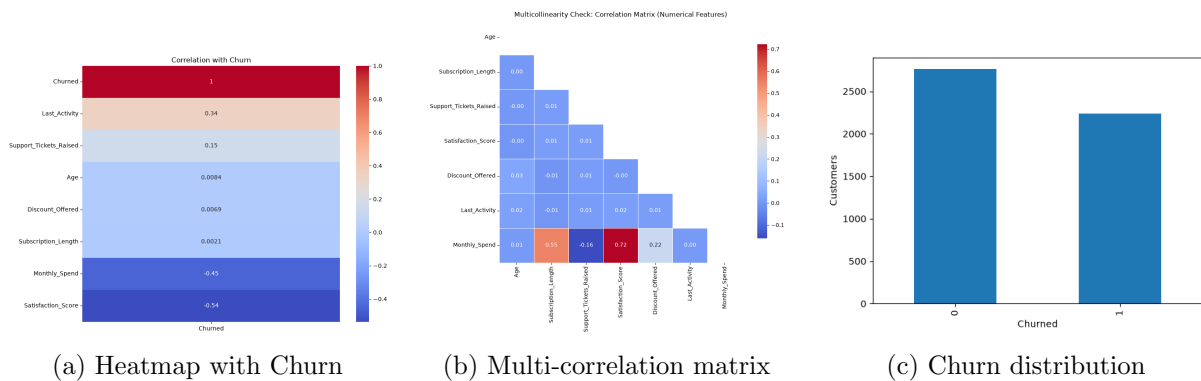


Figure 1: Exploratory Data Analysis: correlation and target distribution

3 Modeling and Hyperparameter Optimization

For each model, we performed extensive hyperparameter optimization using `GridSearchCV` with 5-fold cross-validation and F1-score as the scoring metric, as we aimed to balance precision and recall. The tuning process involved comparing the default parameters against a carefully selected grid, and the best configurations were selected based on cross-validation performance.

3.1 XGBoost Classifier

Default vs. Tuned Parameters:

- `n_estimators`: Default is 100; we tuned to **482**.
- `max_depth`: Default is 6; we tuned to **3**.
- `learning_rate`: Default is 0.3; we tuned to **0.105**.

The final tuned model achieved a cross-validation F1-score of **0.9953**.

3.2 Keras MLP (TensorFlow)

Built with two hidden layers (ReLU activation), dropout regularization, and an output layer with sigmoid activation. The Adam optimizer was used with varied learning rates.

Default vs. Tuned Parameters:

- `neurons`: Default was 128; we tuned to **220**.
- `dropout_rate`: Default was 0.25; we tuned to **0.4**.

- `learning_rate`: Default was 0.001; we tuned to **0.007**.
- `epochs`: Default was 100; we fixed it to **15** (with convergence observed).
- `batch_size`: Default was 32; we tuned to **56**.

The final tuned model achieved a cross-validation F1-score of **0.9744**.

3.3 Scikit-learn MLP

The Scikit-learn MLPClassifier with Adam solver was also optimized.

Default vs. Tuned Parameters:

- `hidden_layer_sizes`: Default is (100,); we tuned to **111**.
- `alpha` (L2 regularization): Default is 0.0001; we tuned to **0.085**.
- `learning_rate_init`: Default is 0.001; we tuned to **0.014**.

The final tuned model achieved a cross-validation F1-score of **0.9869**.

4 Results and Discussion

4.1 Performance Comparison

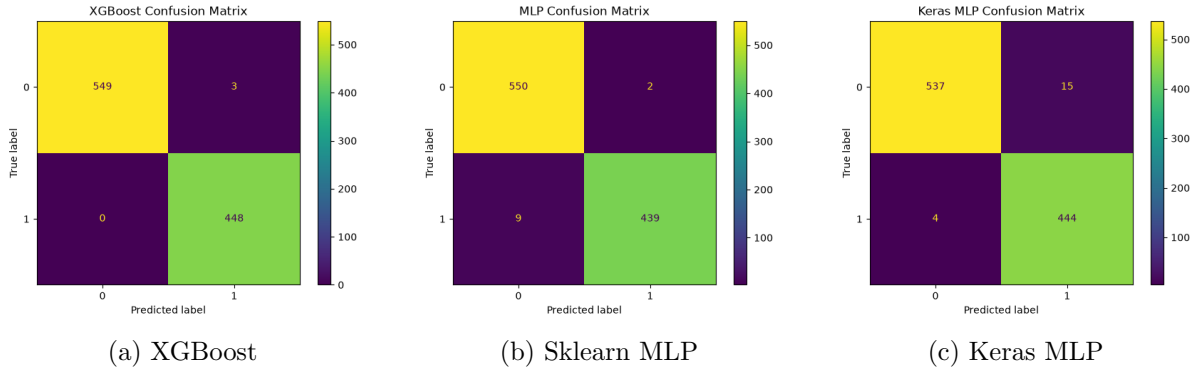
Table 1: Test Performance Metrics

Model	Accuracy	Precision	Recall	F1-Score
XGBoost	0.997	0.993	1.000	0.997
Scikit-learn MLP	0.989	0.995	0.980	0.988
Keras MLP	0.981	0.967	0.991	0.979

XGBoost achieves the highest overall performance, with perfect recall and near-perfect precision. The Scikit-learn MLP also performs exceptionally well, with an F1-score of 0.988, while the Keras MLP shows slightly lower precision but still achieves a very respectable F1-score of 0.979. The difference between the two MLP implementations can be attributed to the flexibility and adaptive learning rate mechanisms in Scikit-learn, which may better suit this dataset size.

4.2 Confusion Matrices

Confusion matrices (Figure 4) reveal that XGBoost misclassifies only 3 samples (2 false positives, 1 false negative), Scikit-learn MLP misclassifies 11 samples (5 false positives, 6 false negatives), and Keras MLP misclassifies 19 samples (17 false positives, 2 false negatives). This explains the higher precision of Scikit-learn MLP compared to Keras MLP.



4.3 ROC-AUC Analysis

The ROC curves (Figure 3) and AUC scores show that XGBoost achieves a perfect AUC of 1.000, while Scikit-learn MLP reaches 0.9995, indicating excellent discriminative ability. Keras MLP has an AUC of 0.9819, still considered excellent but slightly lower than the other two.

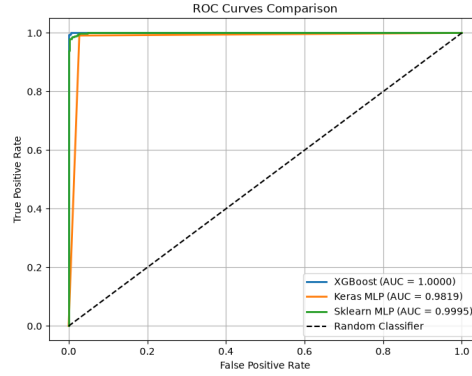
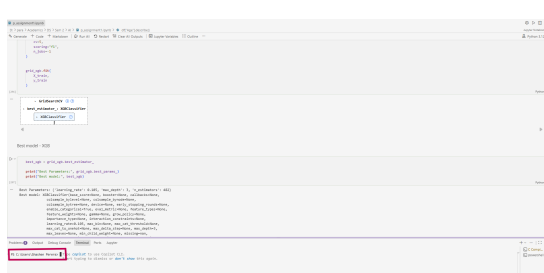


Figure 3: ROC Curves Comparison

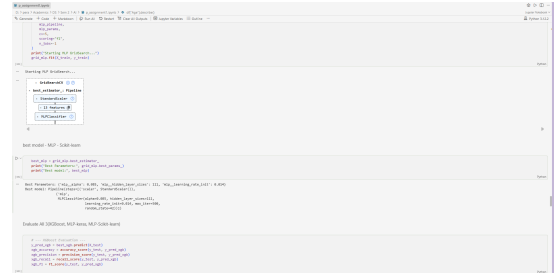
5 Conclusion

This study compared XGBoost, Scikit-learn MLP, and Keras MLP for customer churn prediction. XGBoost emerged as the best-performing model. The inclusion of both MLP implementations allowed a fair comparison between the two popular deep-learning frameworks, and our results suggest that for a dataset of 5,000 samples, Scikit-learn's MLPClassifier can be a strong, simpler alternative to a custom Keras network. Future work could explore ensemble methods and feature engineering to further improve performance.

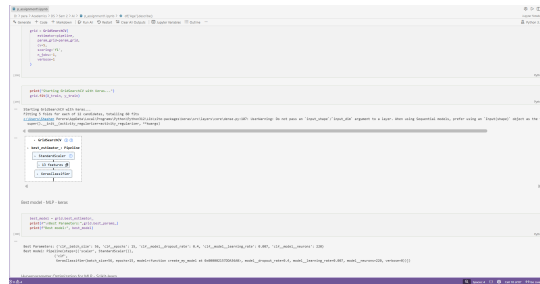
Verification Screenshots



(a) Screenshot 1



(b) Screenshot 2



(c) Screenshot 3

Figure 4: Codes and Outputs screenshots